Requirements Gathering

Comprehensive requirements gathering is the foundation of successful specification-driven development. This section guides you through structured techniques to capture detailed functional and non-functional requirements for your MyCoolChatbot-KiroSpec application.

Exercise 3.1: Functional Requirements Specification

Create detailed functional requirements using structured analysis techniques:

```
Create a comprehensive functional requirements specification for our serverless MyCoolChatbot-KiroSpec application:

Use the MCP servers and Agent Steering to research current best practices, then create:**

**Core Functional Requirements**:

**FR1: User Interface and Interaction**
- FR1.1: The system shall provide a web-based chat interface accessible via modern web browsers
- FR1.2: The interface shall display conversation history with clear distinction between user and bot messages
- FR1.3: The system shall provide real-time typing indicators during message processing
- FR1.4: The interface shall support message input via text with send button and Enter key
- FR1.5: The system shall display error messages and retry options for failed interactions

**FR2: Chat Processing and AI Integration**
- FR2.1: The system shall process user text messages and generate contextually appropriate responses
- FR2.2: The system shall integrate with Amazon Bedrock for AI-powered response generation
- FR2.3: The system shall maintain conversation context for natural, coherent interactions
- FR2.4: The system shall support conversation history retrieval and display
- FR2.5: The system shall handle multiple concurrent user sessions independently

**FR3: User Management and Authentication**
- FR3.1: The system shall support user registration with email and password
- FR3.2: The system shall authenticate users and maintain secure sessions
- FR3.3: The system shall provide password reset and account recovery functionality
- FR3.4: The system shall support user profile management and preferences
- FR3.5: The system shall implement proper session timeout and security controls

**For each requirement, include:**
- Detailed acceptance criteria
- User stories and use cases
- Input/output specifications
- Error handling requirements
- Performance expectations

Ask Kiro to research current chatbot best practices using the AWS documentation MCP server and incorporate those insights into the requirements.
```

Expected Outcome

- Comprehensive functional requirements with detailed acceptance criteria
- User stories and use cases for each major feature
- Clear input/output specifications for all interactions
- Error handling and edge case requirements
- Integration of current best practices from AWS documentation

Exercise 3.2: Non-Functional Requirements Specification

Define comprehensive non-functional requirements covering performance, security, and scalability:

```
Create detailed non-functional requirements specification leveraging AWS best practices:

**Performance Requirements**:
- NFR1.1: The system shall respond to user messages within 2 seconds under normal load (95th percentile)
- NFR1.2: The frontend shall achieve First Contentful Paint (FCP) within 1.5 seconds
- NFR1.3: The system shall support at least 1000 concurrent users without degradation
- NFR1.4: API endpoints shall have response times under 500ms for 99% of requests
- NFR1.5: The system shall maintain performance during traffic spikes up to 10x normal load

**Scalability Requirements**:
- NFR2.1: The system shall automatically scale to handle varying loads without manual intervention
- NFR2.2: Infrastructure shall support horizontal scaling of all components
- NFR2.3: The system shall handle graceful degradation under extreme load conditions
- NFR2.4: Database operations shall scale to support growing user base and conversation history
- NFR2.5: CDN and caching strategies shall optimize global content delivery

**Security Requirements**:
- NFR3.1: All data transmission shall use TLS 1.2 or higher encryption
- NFR3.2: User authentication shall implement industry-standard security practices
- NFR3.3: The system shall protect against common web vulnerabilities (OWASP Top 10)
- NFR3.4: API access shall be secured with proper authentication and authorization
- NFR3.5: User data shall be encrypted at rest using AES-256 encryption

**Reliability and Availability Requirements**:
- NFR4.1: The system shall maintain 99.9% uptime (8.76 hours downtime per year maximum)
- NFR4.2: The system shall implement automatic failover and recovery mechanisms
- NFR4.3: Data backup and recovery procedures shall ensure RPO of 1 hour and RTO of 4 hours
- NFR4.4: The system shall gracefully handle third-party service failures (Bedrock, AWS services)
- NFR4.5: Monitoring and alerting shall provide proactive issue detection and resolution

Use the AWS documentation MCP server to research current AWS well-Architected Framework principles and incorporate them into these requirements.
```

Expected Outcome

- Comprehensive non-functional requirements aligned with AWS Well-Architected Framework
- Specific, measurable performance and scalability targets
- Detailed security requirements following industry best practices
- Reliability and availability requirements with clear metrics
- Integration of current AWS best practices and recommendations

Exercise 3.3: User Experience and Accessibility Requirements

Define comprehensive UX and accessibility requirements following modern standards:

```
Create detailed user experience and accessibility requirements:

**User Experience Requirements**:
- UXR1.1: The interface shall be intuitive and require no training for basic chat functionality
- UXR1.2: The system shall provide clear visual feedback for all user actions
- UXR1.3: Error messages shall be user-friendly and provide actionable guidance
- UXR1.4: The interface shall be responsive and work seamlessly on mobile, tablet, and desktop
- UXR1.5: Loading states and progress indicators shall keep users informed during processing

**Accessibility Requirements (WCAG 2.1 AA Compliance)**:
- AR1.1: All interactive elements shall be keyboard accessible with logical tab order
- AR1.2: Color contrast shall meet minimum 4.5:1 ratio for normal text and 3:1 for large text
- AR1.3: All images and non-text content shall have appropriate alternative text
- AR1.4: The interface shall be compatible with screen readers and assistive technologies
- AR1.5: Users shall be able to resize text up to 200% without loss of functionality

**Internationalization Requirements**:
- IR1.1: The interface shall support multiple languages with proper text direction
- IR1.2: Date, time, and number formats shall adapt to user locale preferences
- IR1.3: The system shall handle Unicode characters and emoji in chat messages
- IR1.4: Text expansion for translations shall not break layout or functionality
- IR1.5: Cultural considerations shall be incorporated into AI response generation

**Mobile and Cross-Platform Requirements**:
- MPR1.1: Touch targets shall be minimum 44px × 44px for optimal mobile interaction
- MPR1.2: The interface shall work offline with appropriate messaging and queue functionality
- MPR1.3: Progressive Web App features shall enable installation and native-like experience
- MPR1.4: The system shall work consistently across all major browsers and versions
- MPR1.5: Performance shall be optimized for slower mobile networks and devices

Research current accessibility best practices and mobile UX patterns using the web search MCP server to ensure requirements reflect the latest standards.
```

Expected Outcome

- Comprehensive UX requirements focused on usability and user satisfaction
- Detailed accessibility requirements ensuring WCAG 2.1 AA compliance
- Internationalization support for global user base
- Mobile-first design requirements with cross-platform compatibility
- Integration of current UX and accessibility best practices

Exercise 3.4: Integration and Compliance Requirements

Define requirements for external integrations and regulatory compliance:

```
Create comprehensive integration and compliance requirements:

**AWS Services Integration Requirements**:
- AIR1.1: Amazon Bedrock integration shall support multiple AI models with fallback capabilities
- AIR1.2: CloudWatch integration shall provide comprehensive logging and monitoring
- AIR1.3: S3 integration shall handle file storage with proper lifecycle management
- AIR1.4: API Gateway integration shall provide rate limiting, throttling, and security
- AIR1.5: Lambda functions shall implement proper error handling and retry logic

**Third-Party Integration Requirements**:
- TIR1.1: External API integrations shall implement circuit breaker patterns for resilience
- TIR1.2: Webhook handling shall include proper validation and security measures
- TIR1.3: Payment processing integration shall meet PCI DSS compliance requirements (if applicable)
- TIR1.4: Analytics integration shall respect user privacy and consent preferences
- TIR1.5: Social media integrations shall follow platform-specific security guidelines

**Data Privacy and Compliance Requirements**:
- DPR1.1: GDPR compliance shall include user consent management and data portability
- DPR1.2: Data retention policies shall automatically delete user data per regulatory requirements
- DPR1.3: User data processing shall implement privacy by design principles
- DPR1.4: Cross-border data transfer shall comply with applicable data protection laws
- DPR1.5: Audit logging shall track all data access and processing activities

**Security Compliance Requirements**:
- SCR1.1: SOC 2 Type II compliance shall be maintained for all security controls
- SCR1.2: Penetration testing shall be conducted quarterly with remediation tracking
- SCR1.3: Vulnerability management shall include automated scanning and patch management
- SCR1.4: Incident response procedures shall meet regulatory notification requirements
- SCR1.5: Security training and awareness programs shall be implemented for all team members

Use the AWS documentation MCP server to research current compliance frameworks and security best practices for serverless applications.
```

Expected Outcome

- Comprehensive integration requirements for AWS and third-party services
- Detailed compliance requirements covering GDPR, SOC 2, and other regulations
- Security compliance framework with specific controls and procedures
- Data privacy requirements with clear policies and procedures
- Integration of current compliance best practices and regulatory requirements

Exercise 3.5: Requirements Validation and Traceability

Establish requirements validation and traceability processes:

```
Create a comprehensive requirements validation and traceability framework:

**Requirements Validation Process**:
1. **Completeness Validation**: Ensure all functional areas are covered with sufficient detail
2. **Consistency Validation**: Verify requirements don't conflict with each other
3. **Testability Validation**: Confirm all requirements can be objectively tested
4. **Feasibility Validation**: Assess technical and business feasibility of all requirements
5. **Stakeholder Validation**: Obtain approval from all relevant stakeholders

**Traceability Matrix Creation**:
Create a traceability matrix that links:
- Business objectives to functional requirements
- Functional requirements to non-functional requirements
- Requirements to architecture components
- Requirements to test cases and acceptance criteria
- Requirements to implementation tasks and deliverables

**Requirements Management Process**:
- **Change Control**: Establish process for requirement changes and impact analysis
- **Version Control**: Implement versioning for requirements documentation
- **Impact Analysis**: Assess impact of requirement changes on architecture and implementation
- **Approval Workflow**: Define approval process for requirement changes
- **Communication**: Establish process for communicating requirement changes to stakeholders

**Quality Assurance for Requirements**:
- **Review Checklist**: Create checklist for requirements quality review
- **Acceptance Criteria**: Define clear acceptance criteria for each requirement
- **Definition of Done**: Establish what constitutes a complete requirement
- **Metrics and KPIs**: Define metrics for requirements quality and completeness
- **Continuous Improvement**: Process for improving requirements gathering and management

Set up Agent Hooks to automatically validate requirements completeness and consistency when specification files are updated.
```

Expected Outcome

- Comprehensive requirements validation framework ensuring quality and completeness
- Traceability matrix linking requirements to implementation and testing
- Requirements management process with change control and approval workflows
- Quality assurance processes for maintaining requirements integrity
- Automated validation through Agent Hooks integration

Key Deliverables

After completing the Requirements Gathering exercises, you should have:

1. **Comprehensive Functional Requirements**: Detailed specifications for all system functionality
2. **Complete Non-Functional Requirements**: Performance, security, and scalability specifications
3. **UX and Accessibility Requirements**: User-centered design and accessibility compliance
4. **Integration and Compliance Requirements**: External integrations and regulatory compliance
5. **Validation and Traceability Framework**: Processes for maintaining requirements quality

Integration with Advanced Features

Your requirements gathering process leverages:

- **MCP Servers**: For researching current best practices and standards
- **Agent Steering**: For applying consistent requirements formatting and quality
- **Agent Hooks**: For automated validation and documentation updates

Next Steps

With comprehensive requirements gathered and validated, you're ready to design the system architecture. The next section will guide you through creating detailed architecture specifications that directly address your requirements while following AWS best practices and modern design patterns.

The architecture design will serve as the blueprint for all subsequent technical specifications and implementation work.

aws workshop studio

rajmkumar

Event ends in 2 days 8 hours 11 minutes.

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Privacy policy Terms of use Cookie preferences